

[TERUG NAAR DE LANDINGSPAGINA](#)

MAKER MANUALS 04

AI-AUTONOMIE

Onafhankelijk blijven in een wereld vol modellen, platforms en AI-tools.

Formaat

Webeditie in neo brutalism-stijl

Licentie

CC BY-SA 4.0

Status

Alpha-editie 0.4

Maker Manuals 04

Onafhankelijk blijven in een wereld vol modellen,
platforms en AI-tools

Alpha-editie 0.4 - vroege versie in ontwikkeling

Dit boek is bewust een werk in uitvoering: geen eindpunt, maar een versie die beter wordt terwijl het onderwerp beweegt.

Gebaseerd op de nieuwsbrieven van Erwin Blom, met hulp van AI in een Maker Manual omgezet.

Dat betekent: je mag deze tekst delen en bewerken, ook commercieel, zolang je Erwin Blom als bron noemt en afgeleide versies onder dezelfde licentie verspreidt.

INHOUD

1. Hoofdstuk 1. Waarom AI-autonomie ineens belangrijk is
2. Hoofdstuk 2. De nieuwe afhankelijkheid
3. Hoofdstuk 3. Bouw je eigen laag boven de modellen
4. Hoofdstuk 4. Waarom chats geen geheugen mogen zijn
5. Hoofdstuk 5. Bestanden, formats en markdown als machtsmiddel
6. Hoofdstuk 6. Werken met meerdere modellen zonder gek te worden
7. Hoofdstuk 7. Wat houd je zelf, wat huur je in
8. Hoofdstuk 8. Je playbooks en kwaliteitslat vastleggen
9. Hoofdstuk 9. Bouw een klein maar sterk model-onafhankelijk systeem
10. Hoofdstuk 10. Onafhankelijk genoeg is beter dan volledig autonoom

Hoofdstuk 1. Waarom AI-autonomie ineens belangrijk is

De meeste mensen stappen AI binnen via gemak. Dat is logisch. Een chatvenster werkt meteen. Een model helpt met schrijven, samenvatten, zoeken of structureren. Een nieuwe tool belooft snelheid. Een browseragent klikt taken weg. Een app maakt in een paar minuten iets wat vroeger dagen of weken kostte. Het voelt vaak als pure winst.

En in het begin is het dat ook.

Het probleem ontstaat pas later. Niet op de eerste dag dat je een model gebruikt, maar op de honderdste. Niet wanneer je een losse prompt probeert, maar wanneer werkprocessen zich in een platform beginnen vast te zetten. Dan merk je dat je aantekeningen in chats wonen, dat je vaste instructies nergens echt beheerd worden, dat waardevolle context alleen nog in de geschiedenis van een bepaalde tool terug te vinden is en dat je werktempo ongemerkt afhankelijk is geworden van één interface.

Dat is de echte reden waarom dit boek nodig is.

AI-autonomie gaat niet over de droom om nooit meer afhankelijk te zijn van technologiebedrijven. Dat is voor de meeste mensen onhaalbaar, en meestal ook niet nodig. AI-autonomie gaat over iets veel praktischer: hoe voorkom je dat je werk vastloopt zodra een model verandert, een prijs omhoog gaat, een tool verdwijnt of een interface verslechtert?

Die vraag is ineens belangrijk geworden omdat AI sneller beweegt dan gewone software.

Bij traditionele software kon je jaren werken met ongeveer hetzelfde pakket. Er kwamen updates, concurrenten, prijswijzigingen, soms een overname. Dat was vervelend, maar meestal beheersbaar. Bij AI is de grond veel bewegelijker. Modellen worden snel beter of juist tijdelijk slechter op specifieke taken. Interfaces veranderen. Producten worden samengevoegd of uitgekleed. Er verschijnen nieuwe spelers die ineens veel aantrekkelijker zijn dan de koploper van drie maanden eerder. Wie alles in een enkele leverancier stopt, bouwt dus op een basis die veel minder stabiel is dan hij lijkt.

Daar komt nog iets bij.

AI raakt niet alleen je software, maar ook je denkwerk. Dat maakt de afhankelijkheid dieper. Bij een tekstverwerker kun je relatief makkelijk weg. Bij een AI-systeem waarin je toon, context,

werkafspraken, projectkennis en persoonlijke routines zijn gaan wonen, is overstappen veel lastiger. Dan moet je niet alleen bestanden meenemen, maar ook een manier van werken.

Veel gebruikers merken dat nog niet, omdat de opbrengst nu zo zichtbaar is. Een goede chatreactie voelt immers direct nuttig. Een agent die iets voor je uitzoekt, geeft meteen winst. Een tool die een eerste versie van een pagina bouwt, maakt indruk. Maar juist in die fase is het verstandig om alvast een tweede vraag te stellen: waar blijft dit werk eigenlijk?

Staat het in mijn eigen omgeving, of alleen in die van iemand anders?

Dat is geen ideologische vraag. Het is een praktische.

Wie met AI werkt, bouwt namelijk altijd twee dingen tegelijk. Aan de oppervlakte bouw je output: teksten, analyses, tools, schema's, samenvattingen, workflows. Daaronder bouw je nog iets anders op: context, routines, voorkeuren, standaarden, beslisregels en geheugen. Die tweede laag is vaak waardevoller dan het eerste losse resultaat. Als die laag alleen binnen een platform leeft, ben je kwetsbaarder dan je denkt.

Dat betekent niet dat je nu alles lokaal moet draaien of zelf moet hosten. Het betekent wel dat je moet nadenken over wat van jou moet blijven, ook als de motor verandert. In de kern is dit boek dus een pleidooi voor een eigen laag boven de modellen. Niet tegen gemak, maar tegen achteloosheid.

De mensen die de komende jaren het meeste hebben aan AI, zijn waarschijnlijk niet de mensen die het trouwst zijn aan één tool. Het zijn de mensen die slim gebruikmaken van goede modellen en tegelijk genoeg afstand bewaren om te kunnen wisselen. Niet omdat wisselen leuk is, maar omdat overstapruimte rust geeft.

Die rust is meer waard dan ze klinkt.

Wie weet dat notities, bronbestanden, werkinstructies en publicatielagen in eigen beheer zijn, raakt minder snel in paniek door de volgende AI-verschuiving. Je hoeft niet telkens helemaal opnieuw te beginnen. Je hoeft niet elke nieuwe tool als verlosser te zien. Je kunt kiezen op kwaliteit, prijs of gebruiksgemak, zonder dat je hele werkwijze mee omvalt.

Dat is de hoofdgedachte van dit boek:

gebruik de kracht van AI voluit, maar zorg dat wat jij opbouwt niet volledig van die krachtbron afhankelijk wordt.

AI-autonomie is dus geen vlucht uit de moderne tooling. Het is een manier om er vol in te gaan zonder jezelf erin op te sluiten.

Hoofdstuk 2. De nieuwe afhankelijkheid

Afhankelijkheid van technologie is niet nieuw. We leven al jaren in een wereld van platformen, clouds, abonnementen en gesloten ecosystemen. Toch voelt de afhankelijkheid rond AI anders. Dat komt doordat die dieper het werk in kruipt.

Bij oudere software wist je meestal nog goed wat van jou was. Je had een document, een spreadsheet, een mapstructuur, een CMS, een inbox. Misschien was de tool waarin je werkte niet ideaal, maar de grenzen waren duidelijk. Bij AI vervagen die grenzen. Een gesprek levert een idee op, een analyse, een eerste versie, een nieuwe formulering, een beslissing. Later wil je erop terugkomen en merk je dat het nergens echt geland is. Het zit half in een chat, half in je hoofd en half in een tool die al twintig andere dingen onthoudt.

Dat maakt AI-afhankelijkheid gevaarlijker én subtieler.

De eerste vorm van afhankelijkheid zit in gedrag. Je went aan snelheid. Dat is prettig, maar het maakt lui op een specifieke manier. Je stopt met het netjes vastleggen van dingen, omdat je

vertrouwt op de chatgeschiedenis. Je slaat minder tussenversies op. Je vertrouwt erop dat een model later wel weer iets vergelijkbaars kan produceren. Soms klopt dat. Vaak verlies je intussen context die je niet zomaar terugkrijgt.

De tweede vorm zit in structuur. Veel mensen bouwen inmiddels kleine systemen van prompts, custom instructions, agent-workflows, projectmappen binnen tools en losse automatiseringen. Dat is op zich verstandig. Alleen: als al die onderdelen binnen één omgeving blijven hangen, heb je in feite een intern systeem gebouwd in huurgrond.

Daar zit de echte kwetsbaarheid.

Je bent niet afhankelijk omdat een tool goed is. Je bent afhankelijk zodra de overstapkosten hoog worden. Zodra je denkt: ik zou best willen wisselen, maar dan raak ik te veel kwijt. Of: ik zou deze nieuwe tool willen proberen, maar dan moet ik eerst mijn hele context reconstrueren. Of: dit platform is duurder en stroperiger geworden, maar het zit te diep in mijn manier van werken om nu nog weg te kunnen.

Dat moment komt sneller dan veel mensen verwachten.

AI-tools nodigen lock-in ook actief uit. Niet noodzakelijk uit kwade wil, wel uit productlogica. Het is in hun belang dat je meer context aanlevert, meer taken in dezelfde omgeving doet, meer gewoontegedrag ontwikkelt en minder snel weggaat. Daarom bieden ze geheugenfuncties, projectruimtes, agentlagen, bestandskoppelingen en steeds rijkere interfaces. Dat is vaak nuttig. Het maakt je werk ook plakkeriger.

Daarmee komen we bij een belangrijk onderscheid.

Niet elke afhankelijkheid is verkeerd. Sommige afhankelijkheid koop je bewust in omdat die de moeite waard is. Als een model aantoonbaar beter is voor een taak, is het prima om daar zwaar op

te leunen. Het probleem begint pas wanneer je niet meer goed ziet wat je precies hebt uitbesteed en wat je nog bezit.

Veel gebruikers hebben daar geen helder antwoord op.

Ze weten welke tool ze openen. Ze weten vaak niet waar hun werklaag eindigt en waar de toollaag begint. Dat is waarom AI-autonomie geen technisch hobbyproject is, maar een vorm van professioneel zelfbehoud. Je wilt genoeg overzicht houden om te kunnen zeggen:

- dit is mijn bronlaag
- dit zijn mijn vaste formats
- dit zijn mijn werkafspraken
- dit is mijn projectcontext
- dit is vervangbaar
- en dit wil ik nooit alleen in een platform laten zitten

Zodra je dat onderscheid scherper krijgt, verandert je relatie met AI. Je gebruikt het minder als magische ruimte waarin alles gebeurt, en meer als een reeks krachtige motoren die je in je eigen systeem inzet.

Dat klinkt droger dan het voelt. In de praktijk maakt het je werk juist lichter.

Je hoeft minder te onthouden. Je raakt minder kwijt. Je wordt minder nerveus van modelwissels. Je bent minder geneigd om op elke hype te springen, omdat je weet dat je je basis niet telkens hoeft om te gooien. Dat is misschien wel de belangrijkste opbrengst van autonomie: niet heroische onafhankelijkheid, maar kalmte.

Hoofdstuk 3. Bouw je eigen laag boven de modellen

Als je maar één gedachte uit dit boek onthoudt, laat het dan deze zijn: vertrouw niet alleen op de motor, bouw ook je eigen chassis.

De meeste AI-gebruikers beginnen andersom. Ze leren eerst de motor kennen. Ze ontdekken wat een model goed kan, welke agent handig is, welke tool snel voelt, welke interface prettig werkt. Dat is een logische start. Alleen blijft het daar te vaak bij. Daardoor werken ze eigenlijk rechtstreeks op de machine van iemand anders.

Een eigen laag boven de modellen verandert dat.

Met zo'n laag bedoel ik geen ingewikkelde software-architectuur. Voor de meeste mensen is het veel eenvoudiger. Het is de verzameling van dingen die jouw manier van werken draagbaar maakt:

- je mappenstructuur
- je bronbestanden
- je markdownteksten
- je vaste templates
- je promptbibliotheek
- je checklists
- je kwaliteitscriteria
- je projectoverzichten
- je publicatielagen

Dat lijkt misschien banaal. Juist daar zit de kracht.

Wie zijn werk in gewone bestanden en heldere routines organiseert, hoeft minder te vertrouwen op toevallig chatgeheugen. Je kunt je context opnieuw aan een model voeren. Je kunt een andere tool inzetten op hetzelfde materiaal. Je kunt projecten overdragen. Je kunt later nog begrijpen hoe iets tot stand kwam.

Dat maakt je werk minder spectaculair. Het maakt het wel veel sterker.

Een goed voorbeeld is markdown. Niet omdat markdown heilig is, maar omdat het eenvoudig, leesbaar en breed bruikbaar is. Een

tekst in markdown kan in een editor open, in een chat gebruikt worden, naar HTML vertaald worden, in een EPUB terecht komen, in een kennisbank landen of in een publicatieworkflow meedraaien. Hetzelfde geldt in bredere zin voor platte tekst, HTML en goed beheerde mappen. Simpele formats zijn vaak onderschatte autonomie-tools.

Hetzelfde principe geldt voor prompts en instructies.

Veel mensen hebben inmiddels sterke prompts, maar bewaren die slordig. Een paar zitten in een notitieapp, een paar in custom instructions, een paar in oude chats, een paar in het hoofd. Daarmee bouw je geen eigen laag, maar losse afhankelijkheid. Zodra je prompts, beslisregels en kwaliteitslatten als eigen documenten behandelt, wordt het anders. Dan kun je ze onderhouden, verbeteren, hergebruiken en meenemen.

Een eigen laag helpt ook om AI rustiger te gebruiken.

Zonder zo'n laag voelt elk nieuw project alsof je weer vanaf nul begint. Je zoekt context bij elkaar, legt opnieuw uit wat je wilt, verliest tijd aan ruis en bent sneller onder de indruk van tools die vooral goed zijn in hun eerste vijf minuten. Met een eigen laag stap je anders in. Je hebt bronmateriaal, standaardstructuren en herbruikbare context. Daardoor maakt de specifieke tool minder uit.

Dat is precies de bedoeling.

Het doel is niet om modellen onbelangrijk te maken. Het doel is om de macht van modelwissels kleiner te maken. Een goed model blijft waardevol. Een beter model blijft welkom. Alleen rust je hele werkproces er niet meer volledig op.

Je kunt dat zien als een simpele hiërarchie:

1. bovenaan zit jouw werk
2. daaronder jouw structuur

3. daaronder jouw vaste routines
4. pas daarna komen de modellen en tools

Bij veel gebruikers is die volgorde precies omgekeerd. Dan bepaalt de tool hoe werk eruitziet, waar het blijft en hoe het later terug te vinden is. Dat voelt efficiënt totdat je wilt uitbreiden, publiceren, overdragen of overstappen.

Een eigen laag is dus geen extra ballast. Het is wat voorkomt dat je later alles moet terugwinnen uit systemen die daar niet voor gebouwd zijn.

De vraag is niet of je al zo'n laag hebt. De vraag is meestal: hoe bewust is die al?

Hoofdstuk 4. Waarom chats geen geheugen mogen zijn

Chats zijn verleidelijk. Ze voelen direct, snel en slim. Je kunt denken, proberen, herformuleren, vragen stellen, laten herschrijven en meteen verder. Daardoor lijkt een chatvenster al snel op een werkruimte, een notitieblok en een geheugen tegelijk.

Dat is precies waarom het gevaarlijk is.

Chats zijn geweldig als tussenlaag. Ze zijn zwak als eindbestemming.

De eerste reden is eenvoudig: terugvinden is slecht. Zelfs in goede interfaces blijft een chatgeschiedenis een lang, grillig spoor.

Misschien weet je nog ongeveer wanneer iets gezegd werd.

Misschien herinner je je een trefwoord. Misschien niet. En juist omdat een chat zoveel verschillende soorten werk door elkaar bewaart, vervaagt de waarde. Een sterk inzicht staat naast een vluchtig probeersel, een lijstje naast een losse grap, een echte doorbraak naast tien mislukte iteraties.

De tweede reden is dat chats context opsluiten. Een antwoord uit een chat is vaak alleen volledig begrijpelijk binnen het gesprek waarin het ontstond. Haal je het eruit, dan ontbreken nuance, bron, opdracht of eerdere keuzes. Dat maakt chats slechte archieven. Ze bewaren veel, maar ontsluiten weinig.

De derde reden is dat een chatomgeving zelden jouw informatiestructuur volgt. Projecten lopen door elkaar. Thema's raken versnipperd. Belangrijke stukken zijn niet vanzelf gekoppeld aan je bestanden, je publicatielaag of je routines. Wat in een chat waardevol lijkt, is daarom vaak moeilijk terug te brengen naar een systeem waar je echt op kunt bouwen.

Dit zie je in bijna iedere AI-werkpraktijk terug.

Mensen hebben een goede gedachte gehad. Een model gaf een scherpe formulering. Een agent leverde een bruikbare analyse. Een planning werd helder. Een structuur klopte ineens. En een week later is dat alles alweer half verdwenen in de diepte van een tijdlijn.

Dat is geen klein ongemak. Dat is productiviteitsverlies.

Daarom moet je een simpele vuistregel aanhouden:

alles wat later nog waarde heeft, moet uit de chat naar je eigen laag.

Dat kan op allerlei manieren. Naar een markdownbestand. Naar een projectmap. Naar een bronnotitie. Naar een checklist. Naar een manuscript. Naar een lopend document waarin beslissingen en inzichten samenkomen. De precieze vorm is minder belangrijk dan het principe. Wat belangrijk blijft, mag niet in de chat blijven wonen.

Dat vraagt discipline, maar niet heel veel. Je hoeft niet alles op te slaan. Juist niet. Een groot deel van chatwerk is vluchtig en mag dat ook blijven. De kunst is selectief landen. Wat echt bronwaarde,

projectwaarde of hergebruikwaarde heeft, geef je een duidelijke plek.

Daarmee verandert een chat van archief in werkbank. En dat is een veel betere rol.

Een werkbank mag rommelig zijn. Een archief niet.

Zodra je dat verschil serieus neemt, merk je ook dat AI prettiger gaat werken. Je wordt minder afhankelijk van herinnering. Je verliest minder sterke formuleringen. Je bouwt sneller voort op eerder werk. En je kunt andere tools makkelijker voeden, omdat de echte kennis niet in een afgesloten gesprek gevangen zit.

Hoofdstuk 5. Bestanden, formats en markdown als machtsmiddel

Bestanden klinken saai. Dat is precies hun voordeel.

In een wereld die voortdurend nieuwe interfaces, agents en slimme lagen voortbrengt, zijn gewone bestanden een vorm van rust. Ze vragen niet om bewondering. Ze vragen om beheer. En goed beheer is vaak waardevoller dan een spectaculaire nieuwe tool.

De reden is simpel. Wat in een open of eenvoudig format leeft, blijft verplaatsbaar.

Een tekstbestand kun je openen in allerlei programma's. Een markdownbestand kun je lezen zonder de tool die het maakte. HTML kun je publiceren, bewerken en omzetten. Een EPUB is afgeleid maar breed bruikbaar. Een PDF is niet ideaal als bron, wel nuttig als distributielaag. Een nette mapstructuur maakt projecten begrijpelijk zonder dat je een specifiek platform hoeft te openen.

Daarmee worden formats een machtsmiddel.

Niet in de grote politieke zin van het woord, maar in de dagelijkse praktijk. Ze bepalen hoeveel vrijheid je houdt. Een gesloten tool kan best fantastisch zijn. Alleen: als jouw werk er niet zonder kleerscheuren uit kan, betaal je daar later voor. Misschien niet vandaag. Wel zodra je wilt migreren, hergebruiken, samenvoegen, publiceren of automatiseren.

Daarom is het slim om je bronlaag altijd simpeler te houden dan je zichtlaag.

Je website mag mooier zijn dan je bronbestanden. Je publicatie-ervaring mag rijker zijn dan je manuscriptlaag. Je AI-tool mag slimmer ogen dan je mapstructuur. Maar onder dat alles wil je iets hebben dat niet fragiel is. Iets dat leesbaar blijft. Iets dat niet instort zodra een productbeslissing van een leverancier de verkeerde kant op valt.

Markdown is daar vaak een goede kandidaat voor, juist omdat het zo weinig pretenties heeft. Je kunt er boeken in schrijven, notities in bewaren, prompts in beheren, checklists in bouwen en publicatieworkflows aan koppelen. Het dwingt je niet tot een zwaar systeem. Het ondersteunt wel continuïteit.

Hetzelfde geldt voor hoe je bestanden benoemt en rangschikt. Een goede bestandsnaam is geen detail. Een heldere map is geen decoratie. Het zijn kleine autonomie-keuzes. Ze zorgen ervoor dat jij en anderen later nog snappen wat iets is, waar het hoort en hoe het gebruikt kan worden.

Wie dat onderschat, betaalt vaak in vertraging. Niet doordat iets crasht, maar doordat alles net iets lastiger terug te vinden, uit te wisselen en op te schalen is.

Daarom hoort dit hoofdstuk in een boek over autonomie. Niet omdat bestanden spannend zijn, maar omdat ze in stilte de vrijheid van je werk bepalen.

Hoofdstuk 6. Werken met meerdere modellen zonder gek te worden

Niet elk model is overal het beste in. Dat is inmiddels duidelijk genoeg. Het ene model denkt scherper mee, het andere schrijft prettiger, een derde codeert sterker, een vierde is sneller of goedkoper. Die verscheidenheid is waardevol. Ze wordt pas onhandig wanneer je zonder systeem gaat hoppen.

Veel mensen schieten daarin door naar een van twee kanten.

Of ze blijven alles hardnekkig in één tool doen, ook wanneer die aantoonbaar niet de beste keuze is. Of ze springen juist van model naar model en verliezen daardoor ritme, consistentie en overzicht.

Autonomie vraagt om een derde route.

Niet één model voor alles. Ook niet tien modellen door elkaar. Wel: een kleine, bewuste set van uitwisselbare motoren met een eigen werklaag erboven.

Dat betekent bijvoorbeeld dat je best een voorkeursmodel kunt hebben voor redactie, een ander voor productbouw en weer een ander voor snelle research of bulkwerk. Zolang de context, bronbestanden en kwaliteitslat niet in die tools opgesloten raken, is daar weinig mis mee. Sterker nog: het maakt je flexibeler.

De kunst is dus niet modeltrouw, maar modeldiscipline.

Je wilt kunnen zeggen:

- voor dit soort denkwerk gebruik ik meestal dit model
- voor deze technische taak pak ik liever dat model
- voor deze publicatieronde maakt het model weinig uit, omdat mijn bronlaag al goed is

Wie zo werkt, kan veel rustiger wisselen. Een beter model voelt dan als upgrade, niet als verhuizing.

Daarvoor moet je wel een paar dingen vermijden.

Bewaar geen cruciale instructies alleen in custom settings van één platform. Laat projectkennis niet alleen in één chatomgeving ontstaan. Zorg dat goede prompts, stijlregels en werkafspraken ook buiten de tool bestaan. En vertrouw niet op geheugenfuncties als enige contextlaag. Ze kunnen nuttig zijn. Ze mogen nooit je enige fundament zijn.

Dat is de praktische kant van multi-model werken. De strategische kant is minstens zo belangrijk.

Wie met meerdere modellen kan werken, kijkt nuchterder naar de markt. Nieuwe spelers worden dan kansen in plaats van bedreigingen. Prijswijzigingen voelen minder dwingend. Hype verliest een deel van zijn macht. Je hoeft niet steeds meteen over. Je hoeft ook niet halsstarrig te blijven waar je zit. Je kunt vergelijken vanuit een positie van rust.

Dat is autonomie in actie.

Hoofdstuk 7. Wat houd je zelf, wat huur je in

Er bestaat een romantisch idee over autonomie: dat je alles zelf zou moeten bezitten. Je eigen infrastructuur, je eigen hosting, je eigen modellen, je eigen opslag, je eigen tooling. Voor sommige specialisten is dat interessant. Voor de meeste mensen is het vooral ballast.

Volledige controle is duur.

Niet alleen in geld, ook in aandacht. Alles wat je zelf optuigt, moet je onderhouden, beveiligen, begrijpen en blijven verbeteren.

Daarom is autonomie niet hetzelfde als alles zelf doen. Slimme autonomie begint met onderscheidingsvermogen.

De relevante vraag is niet: wat kan ik zelf?

De relevante vraag is: wat móet van mij blijven, en wat mag best gehuurd zijn zolang ik de uitgang kan vinden?

Dat onderscheid helpt enorm.

Wat je meestal zelf wilt houden:

- bronbestanden
- projectstructuur
- eigen kennislagen
- prompts en instructies die veel waarde dragen
- publicatieteksten
- beslisregels en kwaliteitslatten

Wat je vaak prima kunt huren:

- rekenkracht van modellen
- specifieke interfaces
- specialistische apps
- snelle experimenteertools
- distributielagen zolang export mogelijk blijft

Daar tussenin zit het grijze gebied. Een CMS bijvoorbeeld. Een notitieomgeving. Een automatiseringslaag. Een agentplatform. Daar gaat het niet alleen om de tool, maar om de exit-mogelijkheid. Kun je exporteren? Kun je bestanden meenemen? Kun je elders verder? Zijn je belangrijkste routines elders te reconstrueren?

Dat is de goede toets.

Een gehuurde tool is niet gevaarlijk omdat hij niet van jou is. Hij wordt pas riskant als jij niet meer zonder redelijke schade weg kunt.

Daarom is autonomie vaak gewoon goed huurderschap.

Je gebruikt krachtige diensten. Je profiteert van snelheid en innovatie. Maar je plant niet je hele leven in een woning zonder deur.

Hoofdstuk 8. Je playbooks en kwaliteitslat vastleggen

Veel mensen denken bij autonomie vooral aan tools en bestanden. Dat is terecht, maar onvolledig. Een groot deel van je echte werklaag zit niet in documenten, maar in gewoontes. In hoe je iets aanpakt. In hoe je kwaliteit beoordeelt. In welke stappen je altijd zet voordat iets goed genoeg is.

Zolang die kennis alleen in je hoofd zit, ben je minder autonoom dan je denkt.

Dan ben jij zelf namelijk de bottleneck.

Daarom zijn playbooks belangrijk. Niet als bureaucratie, maar als geheugen voor goed werk. Een playbook kan heel klein zijn. Een checklist voor publicatie. Een vaste structuur voor research. Een redactieaanpak. Een werkwijze voor het omzetten van bronmateriaal naar manuscript. Een standaard voor hoe een agent-output beoordeeld wordt.

Zodra je zulke dingen opschrijft, gebeurt er iets belangrijks.

Je maakt je werk overdraagbaar. Aan een ander mens, aan een toekomstig model en ook aan jezelf over drie maanden. Je hoeft niet telkens opnieuw te bedenken wat "goed" eigenlijk betekent. Je maakt je kwaliteitslat expliciet.

Dat is een onderschatte vorm van autonomie.

Niet omdat je dan minder hulp nodig hebt, maar omdat je beter kunt kiezen welke hulp bruikbaar is. Een model dat jouw standaarden niet haalt, valt sneller door de mand. Een agent die de helft overslaat, wordt makkelijker gecorrigeerd. Een nieuwe tool kun je beter beoordelen, omdat je weet welke uitkomst je nodig hebt.

Wie geen expliciete kwaliteitslat heeft, wordt sneller speelbal van de tool. Wat er goed uit ziet, voelt dan al snel goed genoeg. Terwijl autonomie juist begint met een stevige eigen norm.

Hoofdstuk 9. Bouw een klein maar sterk model-onafhankelijk systeem

Een model-onafhankelijk systeem hoeft niet groot te zijn. Sterker nog: het moet liever klein beginnen.

De fout die veel mensen maken, is dat ze autonomie verwarren met compleetheid. Dan willen ze ineens een perfect dashboard, een eigen geheugenlaag, een promptbibliotheek, een publicatiesysteem, een agentnetwerk en een mapstructuur voor alles. Het gevolg is meestal meer ambitie dan werking.

Begin kleiner.

Voor veel mensen is een eerste sterke versie al dit:

- een duidelijke inbox voor losse input
- een bronlaag voor wat bewaard moet blijven
- een projectlaag per lopend onderwerp
- een manuscript- of outputlaag voor wat gepubliceerd wordt
- een paar vaste formats
- een document met werkinstructies of playbooks

Meer heb je vaak niet nodig om al veel rustiger te werken.

Als je die basis hebt, kun je daar modellen bovenop inzetten. Een model helpt dan bij ordenen, samenvatten, structureren, redigeren,

analyseren of bouwen. Maar de ruggengraat van het werk blijft van jou. Daardoor maakt het minder uit welke motor je vandaag gebruikt.

Dat is de praktische vertaling van autonomie:

geen spectaculair systeem wel een systeem dat jou niet gijzelt

Je merkt snel genoeg waar uitbreiding zinvol is. Misschien wil je later een betere zoeklaag. Misschien een agent die wekelijks een bepaalde bron verwerkt. Misschien een publicatieroute naar site, EPUB en PDF. Prima. Als de basis goed is, groeit dat organisch.

De kunst is dus niet om een totaalplatform te dromen. De kunst is om een kleine werklaag te bouwen die bestand is tegen verandering.

Hoofdstuk 10. Onafhankelijk genoeg is beter dan volledig autonoom

Er zijn twee slechte eindbeelden rond AI.

Het eerste is totale overgave: alles in één platform, alles in chats, alles in de cloud van een leverancier, zonder plan B en zonder eigen structuur.

Het tweede is totale zuiverheid: alles zelf willen, alles lokaal, alles open source, alles onder volledige controle, ongeacht de prijs in tijd, aandacht en complexiteit.

Beide zijn onhandig.

Het eerste maakt je kwetsbaar. Het tweede maakt je traag.

Daarom is de beste uitkomst meestal niet volledige autonomie, maar voldoende autonomie. Genoeg om te kunnen bewegen. Genoeg om te kunnen wisselen. Genoeg om niet opnieuw te

hoeven beginnen als de markt verschuift. Genoeg om je werk van jou te laten blijven, ook wanneer de gereedschappen veranderen.

Dat is een nuchtere ambitie. Juist daarom is ze sterk.

Wie onafhankelijk genoeg is, kan de beste motor van dit moment gebruiken zonder zichzelf eraan te verkopen. Je profiteert van snelheid en kwaliteit, maar bewaart je eigen basis. Je kunt huren zonder je hele huis uit handen te geven. Je kunt vernieuwen zonder je geheugen telkens te verliezen.

Misschien is dat wel de volwassen vorm van AI-gebruik.

Niet: welke tool wint?

Maar: hoe richt ik mijn werk zo in dat ik met goede tools kan werken zonder door ze opgesloten te raken?

Dat is waar AI-autonomie uiteindelijk over gaat.

Niet over afstand nemen van de toekomst.

Wel over zorgen dat je je er vrij genoeg in kunt bewegen.

LICENTIE

Deze tekst is beschikbaar onder de Creative Commons-licentie   4.0.

Dat betekent kort gezegd:

- delen mag
- hergebruiken mag
- bewerken mag

- ook commercieel gebruik is toegestaan

Zolang je Erwin Blom als bron noemt en afgeleide versies onder dezelfde licentie deelt.

Meer informatie:

[Creative Commons BY-SA 4.0](#)